

Lecture 11:

Regular Expressions



INFO 1100: Introduction to Programming

so much interesting data is
represented as *strings*

so much interesting data is
represented as *strings*

Books

Emails

Medical
notes

Historical
letters

Poems

Court
records

what if we want to **search**
that string data?

we know how to do simple searches!

we know how to do simple searches!

e.g.

"and" in "Pride and Prejudice"

... but what if we need to do
something more complicated?

... but what if we need to do
something more complicated?

find dialogue in

Books

... but what if we need to do
something more complicated?

find dates in

Historical
letters

... but what if we need to do
something more complicated?

find greetings in
Emails

... but what if we need to do
something more complicated?

find dosages in

Medical
notes

Regular expressions

Regular expressions

or *regex*

Regular expressions

Why are we learning this?

Regular expressions

Why are we learning this?



more complex data
analysis with basic
Python

Regular expressions

Why are we learning this?



more complex data
analysis with basic
Python

designing + scoping
research questions for
the tools we have

Regular expressions

Regular expressions

What are they?

Regular expressions

a pattern describing a
certain amount of text

Writing regex patterns

Writing regex patterns

To match exact text,
just use the text!

Writing regex patterns

but there are also many **special characters** that we can use to create more complex patterns

Writing regex patterns

character classes

match groups of characters

Writing regex patterns

character classes

match groups of characters

[ae]

a or e

Writing regex patterns

character classes

match groups of characters

[a e]

a or e

[0 - 9]

any single digit

Writing regex patterns

character classes

match groups of characters

[a e]

a or e

[0 - 9]

any single digit

[a - z]

lowercase letters

Writing regex patterns

character classes

match groups of characters

[a e]

a or e

[0 - 9]

any single digit

[a - z]

lowercase letters

[A - Z]

uppercase letters

Writing regex patterns

character classes

match groups of characters

[a e]	a or e
[0 - 9]	any single digit
[a - z]	lowercase letters
[A - Z]	uppercase letters
[a - z A - Z 0 - 9]	alphanumeric chars

Writing regex patterns

character classes (***shorthand***)

match groups of characters

Writing regex patterns

character classes (***shorthand***)

match groups of characters

`\d`

single digit

Writing regex patterns

character classes (***shorthand***)

match groups of characters

`\d`

single digit

`\w`

alphanumeric + `_`

Writing regex patterns

character classes (***shorthand***)

match groups of characters

<code>\d</code>	single digit
<code>\w</code>	alphanumeric + <code>_</code>
<code>\s</code>	whitespace

Writing regex patterns

repetition

match the element n times

Writing regex patterns

repetition

match the element n times

{ d }

match d times

Writing regex patterns

repetition

match the element n times

{ d }

match d times

{ d , n }

match d to n times

Writing regex patterns

repetition

match the element n times

{ d }

match d times

{ d , n }

match d to n times

*

match 0+ times

Writing regex patterns

repetition

match the element n times

{ d }	match d times
{ d , n }	match d to n times
*	match 0+ times
+	match 1+ times

Writing regex patterns

repetition

match the element n times

{ d }	match d times
{ d , n }	match d to n times
*	match 0+ times
+	match 1+ times
?	match 0-1 time

Writing regex patterns

anchors

look at a particular position

Writing regex patterns

anchors

look at a particular position

^

match only at the **start** of lines

Writing regex patterns

anchors

look at a particular position

^ match only at the **start** of lines

\$ match only at the **end** of lines

Writing regex patterns

other

does something wacky + special

Writing regex patterns

other

does something wacky + special

. any character (*except newline!*)

Writing regex patterns

other

does something wacky + special

. any character (*except newline!*)

c.t

matches “cat”, “cot”, “c5t”, “c t”, ...

Writing regex patterns

other

does something wacky + special

. any character (*except newline!*)

\ escape next character (*see as text, not regex*)

Writing regex patterns

other

does something wacky + special

. any character (*except newline!*)

\ escape next character (*see as text, not regex*)

Mr\.

matches *only* “Mr.”

Writing regex patterns

other

does something wacky + special

- . any character (*except newline!*)
- \ escape next character (*see as text, not regex*)
- \b a word boundary

Writing regex patterns

other

does something wacky + special

- . any character (*except newline!*)
- \ escape next character (*see as text, not regex*)
- \b a word boundary
- | match one of the patterns on either side

Grouping

Grouping

using parentheses around a
pattern *groups* it

Grouping

$(ab)^+$

matches “ab” 1+ times

Grouping

(Mr | Mrs | Ms) \.

matches “Mr.” or “Mrs.” or “Ms.”

Grouping

allow us to extract parts of a
match *in Python*

Enter in the chat:

**What pattern do you want to
search for?**

Regex in Python

Regex in Python

```
import re
```


Regex in Python

```
import re
```



the module that
lets us use regular
expressions

Regex in Python

```
pattern = r"cat"
```

Regex in Python

```
pattern = r"c(a|o)t"
```

Regex in Python

```
pattern = r"c(a|o)t"
```

raw strings ignore Python
special characters

Regex in Python

I recommend always using them

```
pattern = r"c(a|o)t"
```

raw strings ignore Python
special characters

Regex in Python

helpful functions

Regex in Python

helpful functions

I'll introduce some – I
encourage you to explore them
on your own time

Regex in Python

helpful functions

```
re.search(<pattern>, <string>)
```


Regex in Python

helpful functions

`re.search(<pattern>, <string>)`

match the first instance of
`<pattern> in <string>`

Regex in Python

helpful functions

```
result = re.search("Dracula", "I am Dracula.")
```

Regex in Python

helpful functions

```
result = re.search("Dracula", "I am Dracula.")
```



```
<re.Match object; span=(5, 12), match='Dracula'>
```

Regex in Python

helpful functions

```
result = re.search("Dracula", "I am Dracula.")
```

```
result.string
```



```
"I am Dracula."
```

Regex in Python

helpful functions

```
result = re.search("Dracula", "I am Dracula.")
```

```
result.group()
```



```
"Dracula"
```

Regex in Python

helpful functions

```
result = re.search("Dracula", "I am Dracula.")
```

```
result.span()
```



```
(5, 12)
```

Regex in Python

helpful functions

If the pattern doesn't appear in the string...

```
result = re.search("Mina", "I am Dracula.")
```

it returns None.

Regex in Python

helpful functions

```
re.findall(<pattern>, <string>)
```


Regex in Python

helpful functions

```
re.findall(<pattern>, <string>)
```

find all (non-overlapping)
instances of <pattern> in
<string>

Regex in Python

helpful functions

```
re.split(<pattern>, <string>)
```

Regex in Python

helpful functions

```
re.split(<pattern>, <string>)
```

split <string> on each
instance of <pattern>;
returns a list

Regex in Python

helpful functions

```
re.sub(<pattern>, <replace>,  
      <string>)
```

Regex in Python

helpful functions

```
re.sub(<pattern>, <replace>,  
      <string>)
```

replace all instances of
<pattern> in <string> with
<replace>

Why are regular expressions a
great application of AI tools?

Regex **limitations**

Regex **limitations**

what can't regex do?

Regex **limitations**

matches only **form**
not **meaning**

Regex **limitations**

can't match nested
structures

Regex **limitations**

can't match nested
structures

e.g. matching parentheses

Example 1

```
import re

text = "The cat sat on the mat."
m = re.search(r"[cm]at", text)
print(m.group())
```

Example 2

```
import re

note = "Room 12, Bed 3: Patient is
       47 years old."

numbers = re.findall(r"\d+", note)

print(numbers)

print(numbers[0] + numbers[2])
```

Example 3

```
import re

def get_year(line):
    m = re.search(r"\d{4}", line)
    if m:
        return m.group()
    return None
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/15 @ 11:30am)

- Lab 11
- Readings
Think Python –
11.1-11.4, 18.1
- GRQs